



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7

Тема:

ОСНОВЫ SQL В POSTGRESQL

по дисциплине «Проектирование баз данных»

Студент группы

ИКБО-50-23. Астахов С.П..

(подпись)

Преподаватель

Копылова Я. А.

(подпись)

Москва 2025 г.

Содержание

1. Цели и задача	3
2. Выполнение практической работы.....	4
2.1 Установка ПО.....	4
2.2 Генерация запроса.....	5
2.3 Создание БД по SQL коду.....	6
3. Вывод.....	14

1. Цели и задача

Цель: сформировать навык по созданию БД и заполнению таблиц с помощью Postgres.

Постановка задачи:

1. Установить PostgreSQL;
2. Установить Dbeaver;
3. Сгенерировать запросы CREATE TABLE по Практической работе №6. Показать скриншотами;
4. Заполнить созданные таблицы с помощью запроса INSERT INTO;
5. Вывести на экран все таблицы SELECT * FROM Table1.

Студентческий номер: 23И0641

Код схемы: 45uz

Хост для подключения: 7bd8a-rw.db.pub.dbaas.postgrespro.ru(порт: 5432)

База данных: dbstud

Пароль: 5\$&VK\$%##W

2. Выполнение практической работы

2.1 Установка ПО

В рамках практической работы установим PostgreSQL и DBeaver. На официальном сайте PostgreSQL — <https://www.postgresql.org/download/> можно скачать сам PostgreSQL и его компоненты, например, визуальную версию — pgAdmin. После закачивания у нас появится новая программа(рис.1):

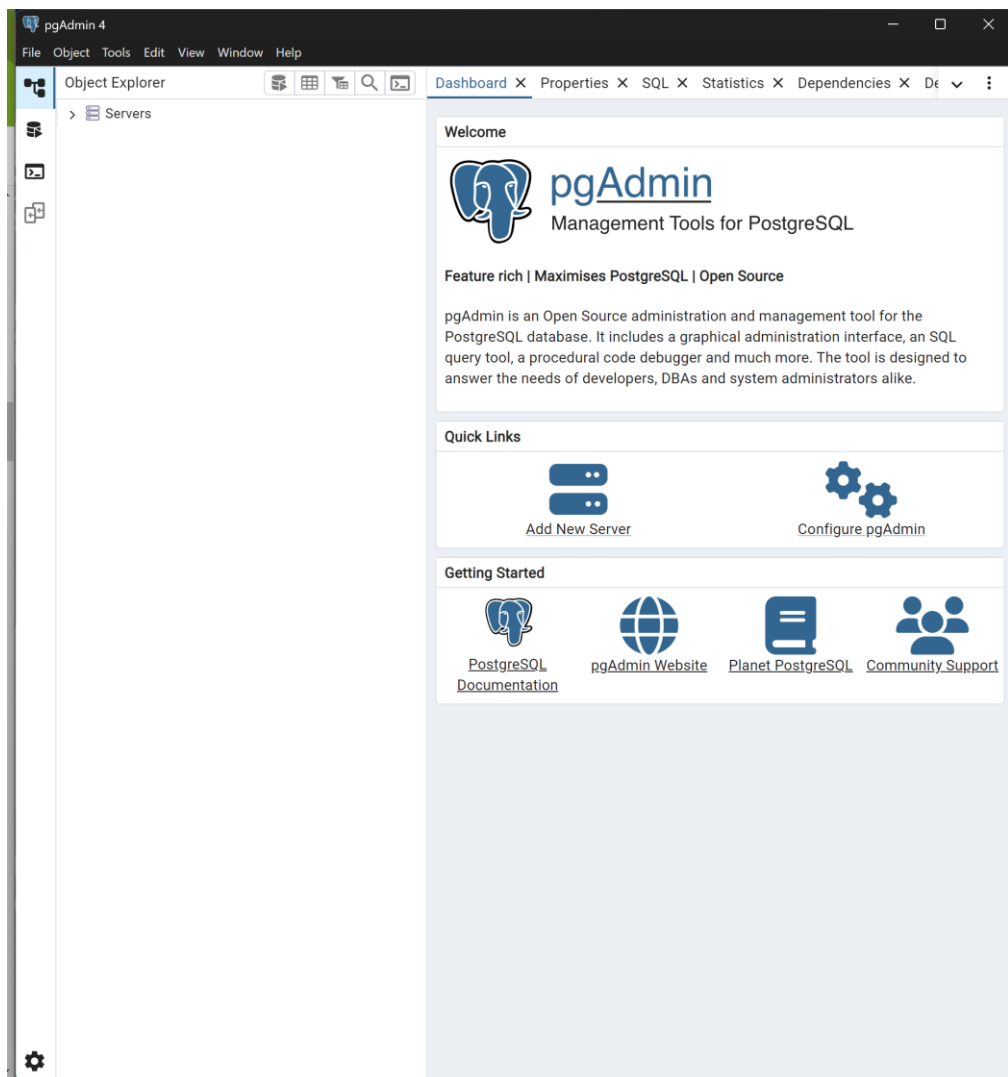


Рисунок 1 – pgAdmin компонент PostgreSQL

Для установки Dbeaver, перейдем на официальный его сайт — <https://dbeaver.io/download/> и выберем установщик «Windows(installer)»(рис. 2):

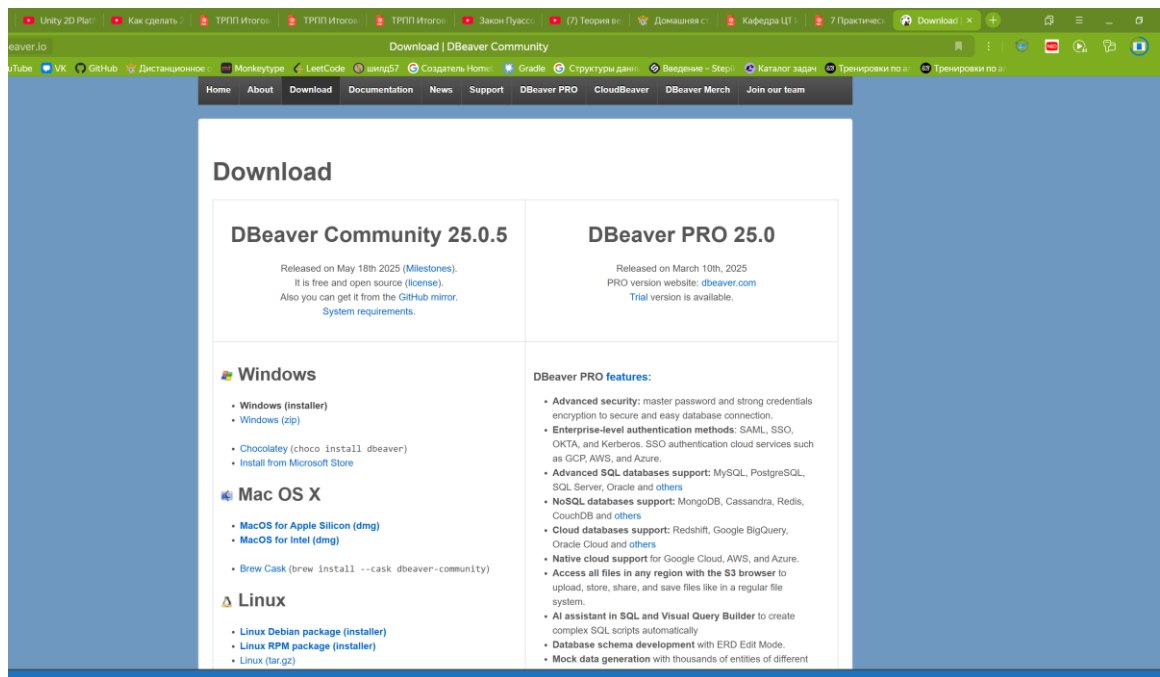


Рисунок 2 – официальный сайт установки Dbeaver

2.2 Генерация запроса

После установки нужного ПО, на сайте ChartDB сгенерируем запрос на получения кода базы данных из 6 практической работы в формате Generic (рис. 3):

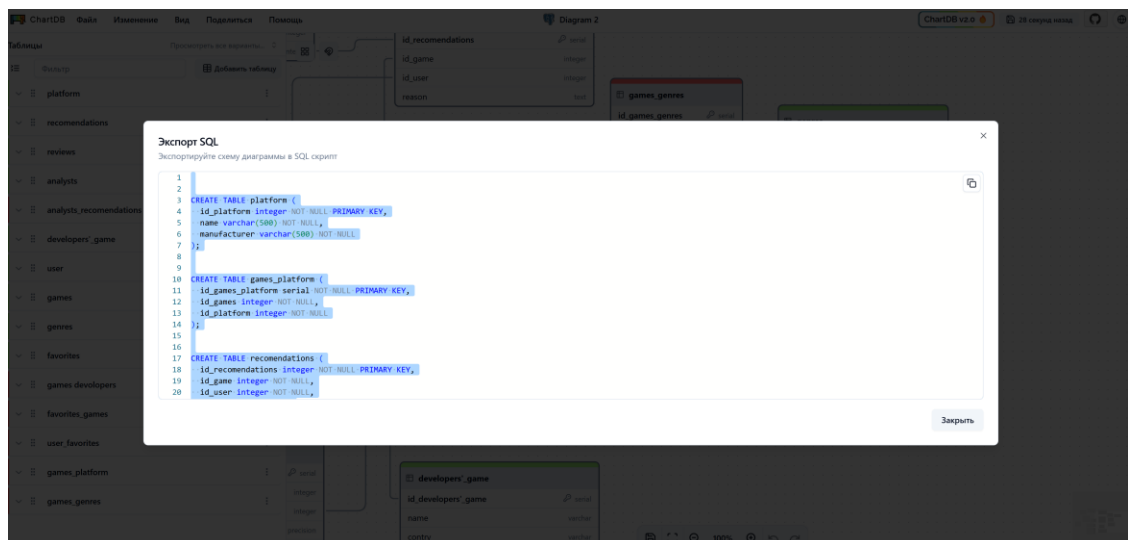


Рисунок 3 – Экспорт SQL кода

2.3 Создание БД по SQL коду и таблиц

Получим код и слегка переделаем его для корректности:

```
CREATE TABLE genres (  
  id_genres serial NOT NULL PRIMARY KEY,  
  name varchar(80) NOT NULL  
);  
  
CREATE TABLE developers_game (  
  id_developers_game serial NOT NULL PRIMARY KEY,  
  name varchar(80) NOT NULL,  
  country varchar(80) NOT NULL  
);  
  
CREATE TABLE platform (  
  id_platform serial NOT NULL PRIMARY KEY,  
  name varchar(50) NOT NULL,  
  manufacturer varchar(80) NOT NULL  
);  
  
CREATE TABLE user_ (  
  id_user serial NOT NULL PRIMARY KEY,  
  name varchar(60) NOT NULL,  
  surname varchar(60) NOT NULL,  
  search_queries varchar(500) NOT NULL,  
  email varchar(80) NOT NULL,  
  date_registration date NOT NULL  
);  
  
CREATE TABLE games (  
  id_game serial NOT NULL PRIMARY KEY,  
  id_developer integer NOT NULL,  
  id_genres integer NOT NULL,  
  name varchar(350) NOT NULL,  
  release_date date NOT NULL,  
  ranking float8 NOT NULL  
);  
  
CREATE TABLE games_platform (  
  id_games_platform serial NOT NULL PRIMARY KEY,  
  id_games integer NOT NULL,  
  id_platform integer NOT NULL  
);  
  
CREATE TABLE favorites (  
  id_favorites serial NOT NULL PRIMARY KEY,  
  age_of_the_duplicate date NOT NULL  
);  
  
CREATE TABLE reviews (  
  id_review serial NOT NULL PRIMARY KEY,  
  id_user integer NOT NULL,  
  id_game integer NOT NULL,  
  ranking float8 NOT NULL,  
  text_ranking text NOT NULL,  
  date date NOT NULL  
);  
  
CREATE TABLE user_favorites (  
  id_user_favorites serial NOT NULL PRIMARY KEY,  
  id_user integer NOT NULL,  
  id_game integer NOT NULL,  
  ranking float8 NOT NULL,  
  text_ranking text NOT NULL,  
  date date NOT NULL  
);
```

```

    id_user_favorites serial NOT NULL PRIMARY KEY,
    id_user integer NOT NULL,
    id_favorites integer NOT NULL
);

CREATE TABLE games_genres (
    id_games_genres serial NOT NULL PRIMARY KEY,
    id_games integer NOT NULL,
    id_genres integer NOT NULL
);

CREATE TABLE recommendations (
    id_recommendations serial NOT NULL PRIMARY KEY,
    id_game integer NOT NULL,
    id_user integer NOT NULL,
    reason text NOT NULL
);

CREATE TABLE analysts (
    id_analyst serial NOT NULL PRIMARY KEY,
    name varchar(80) NOT NULL,
    role varchar(100) NOT NULL
);

CREATE TABLE analysts_recommendations (
    id_analysts_recommendations serial NOT NULL PRIMARY KEY,
    id_analysts integer NOT NULL,
    id_recommendations integer NOT NULL
);

CREATE TABLE favorites_games (
    id_favorites_games serial NOT NULL PRIMARY KEY,
    id_favorites integer NOT NULL,
    id_games integer NOT NULL
);

CREATE TABLE games_developers (
    id_games_developers serial NOT NULL PRIMARY KEY,
    id_games integer NOT NULL,
    id_developers integer NOT NULL
);

ALTER TABLE games ADD CONSTRAINT games_id_genres_fk FOREIGN KEY (id_genres) REFERENCES genres(id_genres);
ALTER TABLE games ADD CONSTRAINT games_id_developer_fk FOREIGN KEY (id_developer) REFERENCES developers_game(id_developers_game);

ALTER TABLE games_platform ADD CONSTRAINT games_platform_id_games_fk FOREIGN KEY (id_games) REFERENCES games(id_game);
ALTER TABLE games_platform ADD CONSTRAINT games_platform_id_platform_fk FOREIGN KEY (id_platform) REFERENCES platform(id_platform);

ALTER TABLE reviews ADD CONSTRAINT reviews_id_game_fk FOREIGN KEY (id_game) REFERENCES games(id_game);
ALTER TABLE reviews ADD CONSTRAINT reviews_id_user_fk FOREIGN KEY (id_user) REFERENCES user(id_user);

ALTER TABLE user_favorites ADD CONSTRAINT user_favorites_id_user_fk FOREIGN KEY (id_user) REFERENCES user(id_user);
ALTER TABLE user_favorites ADD CONSTRAINT user_favorites_id_favorites_fk FOREIGN KEY (id_favorites) REFERENCES favorites(id_favorites);

```

```

ALTER TABLE games_genres ADD CONSTRAINT games_genres_id_games_fk FOREIGN KEY
(id_games) REFERENCES games(id_game);
ALTER TABLE games_genres ADD CONSTRAINT games_genres_id_genres_fk FOREIGN KEY
(id_genres) REFERENCES genres(id_genres);

ALTER TABLE recommendations ADD CONSTRAINT recommendations_id_game_fk FOREIGN KEY
(id_game) REFERENCES games(id_game);
ALTER TABLE recommendations ADD CONSTRAINT recommendations_id_user_fk FOREIGN KEY
(id_user) REFERENCES user(id_user);

ALTER TABLE favorites_games ADD CONSTRAINT favorites_games_id_favorites_fk FOREIGN
KEY (id_favorites) REFERENCES favorites(id_favorites);
ALTER TABLE favorites_games ADD CONSTRAINT favorites_games_id_games_fk FOREIGN KEY
(id_games) REFERENCES games(id_game);

ALTER TABLE games_developers ADD CONSTRAINT games_developers_id_games_fk FOREIGN
KEY (id_games) REFERENCES games(id_game);
ALTER TABLE games_developers ADD CONSTRAINT games_developers_id_developers_fk FOR-
EIGN KEY (id_developers) REFERENCES developers_game(id_developers_game);

ALTER TABLE analysts_recommendations ADD CONSTRAINT analysts_recommendations_id_an-
alysts_fk FOREIGN KEY (id_analysts) REFERENCES analysts(id_analyst);
ALTER TABLE analysts_recommendations ADD CONSTRAINT analysts_recommenda-
tions_id_recommendations_fk FOREIGN KEY (id_recommendations) REFERENCES recommenda-
tions(id_recommendations);

```

Далее мы зайдем в DBeaver и создадим новое соединение. Выберем новым типом соединения PostgreSQL (рис. 4):

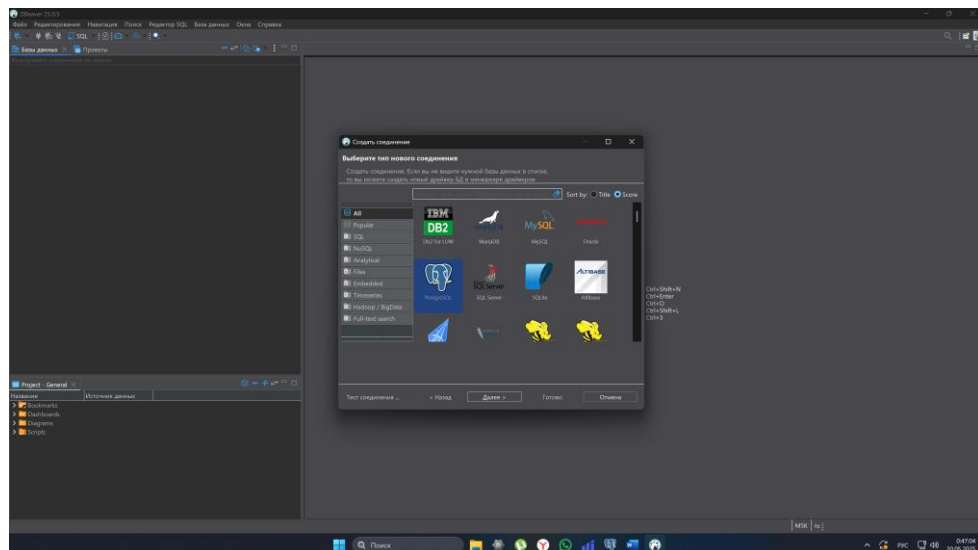


Рисунок 4 – Новое соединение PostgreSQL

Далее нас попросят ввести хост, базу данных и личный пароль для продолжения (рис. 5):

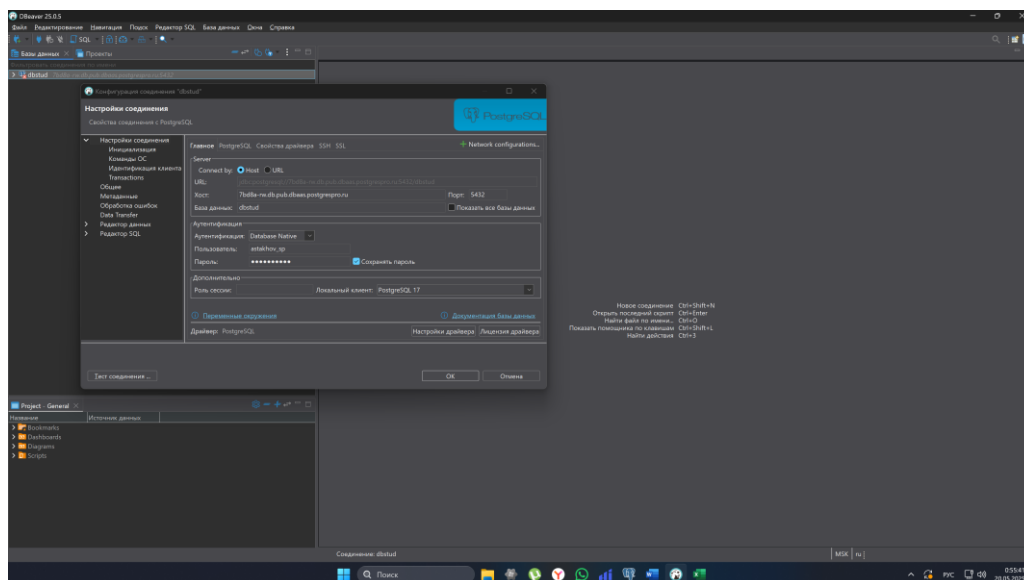


Рисунок 5 – Ввод данных в DBeaver для настройки соединения
И еще установим драйверы (рис. 6):

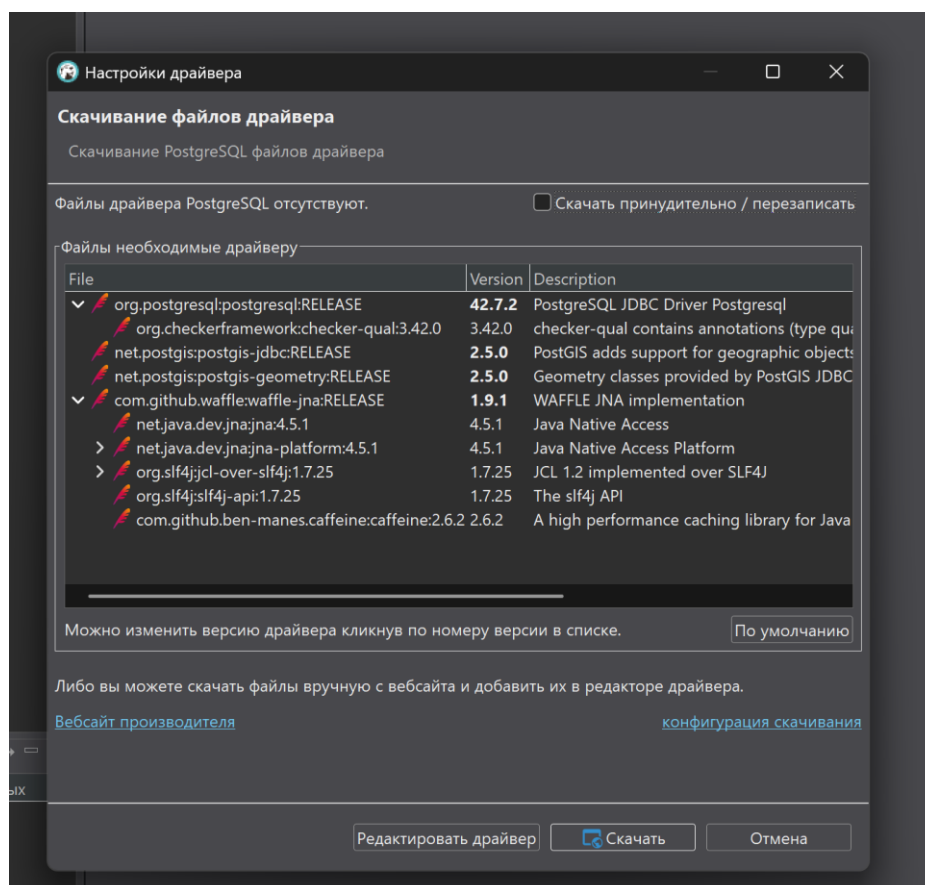
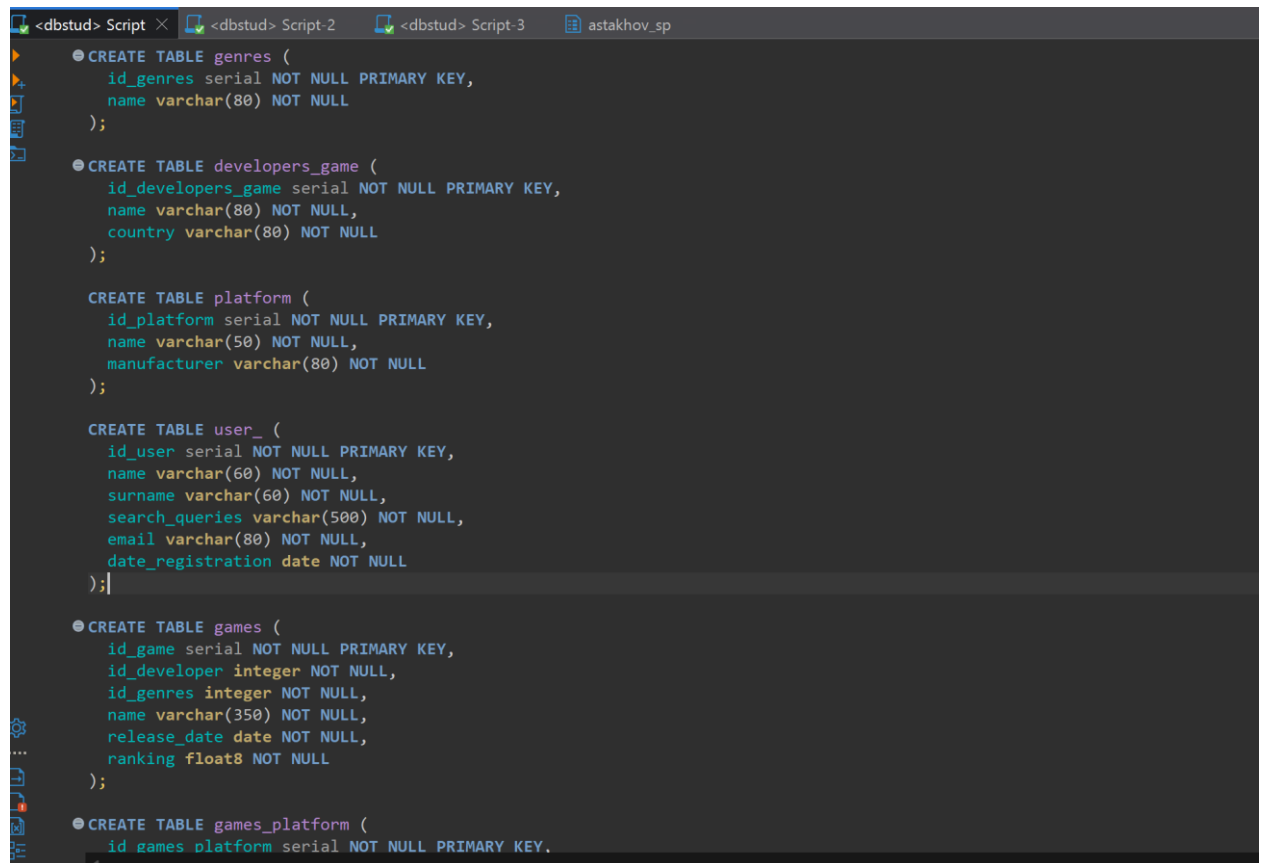


Рисунок 6 – Скачка Драйверов

Теперь у нас есть созданная пустая база данных. Чтоб ее заполнить, мы воспользуемся автоматическим созданием таблиц в редакторе SQL при помощи контекстного меню базы данных. В открывшемся окне редактирования SQL скриптов необходимо вставить ранее выгруженный из

ChartDB отредактированный код (рис. 7):

The image shows a screenshot of a code editor with multiple tabs. The active tab is titled '<dbstud> Script'. It contains several SQL 'CREATE TABLE' statements. The tables being created are: 'genres' with columns 'id_genres' (serial, primary key) and 'name' (varchar(80)); 'developers_game' with columns 'id_developers_game' (serial, primary key), 'name' (varchar(80)), and 'country' (varchar(80)); 'platform' with columns 'id_platform' (serial, primary key), 'name' (varchar(50)), and 'manufacturer' (varchar(80)); 'user_' with columns 'id_user' (serial, primary key), 'name' (varchar(60)), 'surname' (varchar(60)), 'search_queries' (varchar(500)), 'email' (varchar(80)), and 'date_registration' (date); 'games' with columns 'id_game' (serial, primary key), 'id_developer' (integer), 'id_genres' (integer), 'name' (varchar(350)), 'release_date' (date), and 'ranking' (float8); and 'games_platform' with column 'id_games_platform' (serial, primary key). The code is color-coded with syntax highlighting. The editor has a dark theme and a sidebar on the left with icons for file explorer, search, and other tools.

```
● CREATE TABLE genres (  
    id_genres serial NOT NULL PRIMARY KEY,  
    name varchar(80) NOT NULL  
);  
  
● CREATE TABLE developers_game (  
    id_developers_game serial NOT NULL PRIMARY KEY,  
    name varchar(80) NOT NULL,  
    country varchar(80) NOT NULL  
);  
  
CREATE TABLE platform (  
    id_platform serial NOT NULL PRIMARY KEY,  
    name varchar(50) NOT NULL,  
    manufacturer varchar(80) NOT NULL  
);  
  
CREATE TABLE user_ (  
    id_user serial NOT NULL PRIMARY KEY,  
    name varchar(60) NOT NULL,  
    surname varchar(60) NOT NULL,  
    search_queries varchar(500) NOT NULL,  
    email varchar(80) NOT NULL,  
    date_registration date NOT NULL  
);  
  
● CREATE TABLE games (  
    id_game serial NOT NULL PRIMARY KEY,  
    id_developer integer NOT NULL,  
    id_genres integer NOT NULL,  
    name varchar(350) NOT NULL,  
    release_date date NOT NULL,  
    ranking float8 NOT NULL  
);  
  
● CREATE TABLE games_platform (  
    id_games_platform serial NOT NULL PRIMARY KEY.
```

Рисунок 7 - Запросы для создания таблиц

И так, после исполнения SQL скрипта, мы получим базу данных, заполненную таблицами, которые мы описывали в прошлых практиках. База данных студента будет выглядеть так:

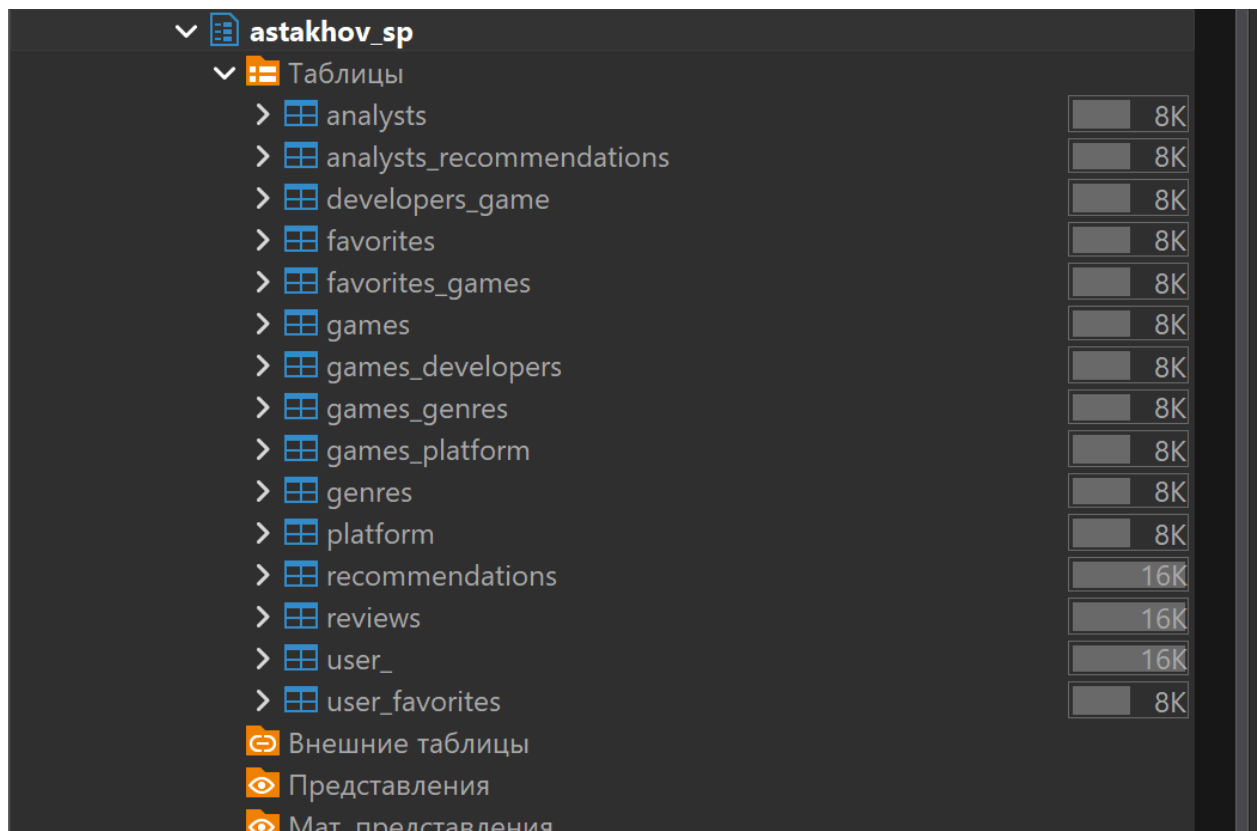


Рисунок 8 - Отображение созданных таблиц

Вывод диаграммы:



Рисунок 9 – Структура диаграммы

2.4 Заполнение таблиц

Далее необходимо заполнить созданные таблицы. Создадим новый редактор SQL так же, как это делалось ранее. В созданном редакторе необходимо написать команды для заполнения таблиц. Заполним таблицы с помощью данного шаблона команды:

insert into <название таблицы> (<имена столбцов через запятую>) values (<значение для заполнения столбцов в том же порядке, что и имена столбцов>);

Таким образом, заполним таблицы (рис. 9):

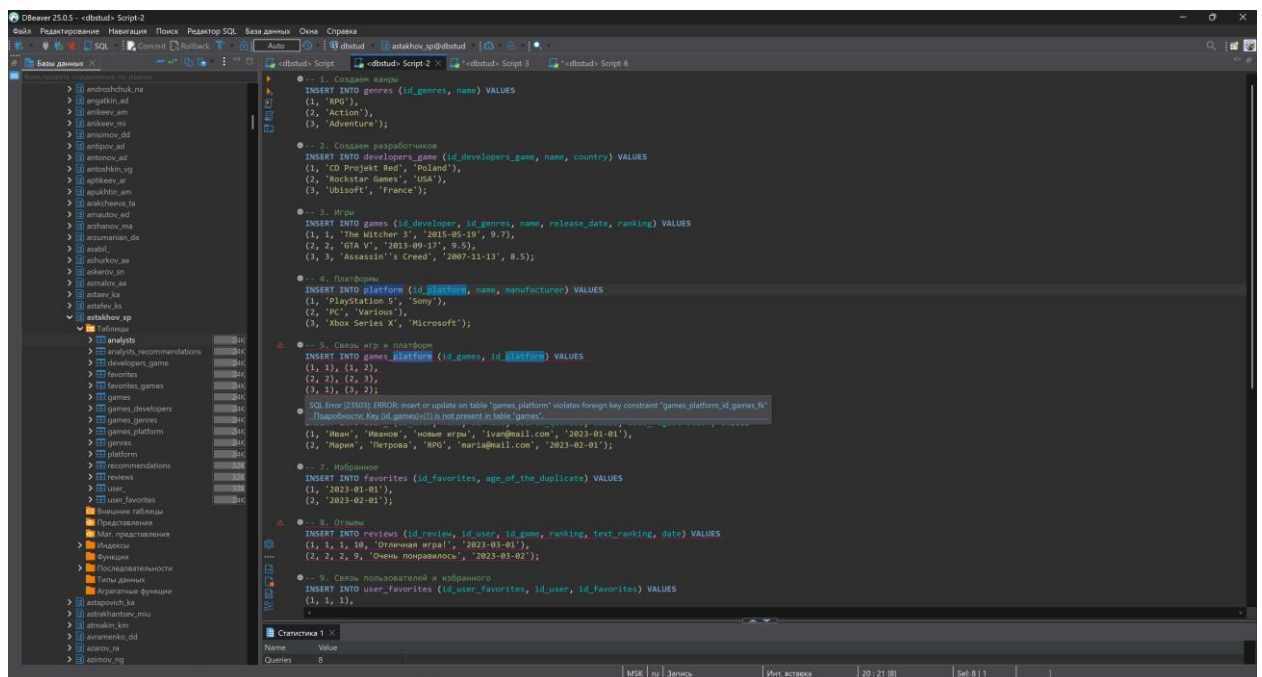


Рисунок 10 - Заполнение таблиц

2.5 Вывод данных в таблице

Наконец, для просмотра заполненных таблиц необходимо создать еще один редактор SQL. Для вывода данных из таблицы используется команда SELECT, обладающая большой гибкостью при использовании. Шаблон команды:

SELECT <Список столбцов через запятую> FROM <Имя таблицы>;

Получим:

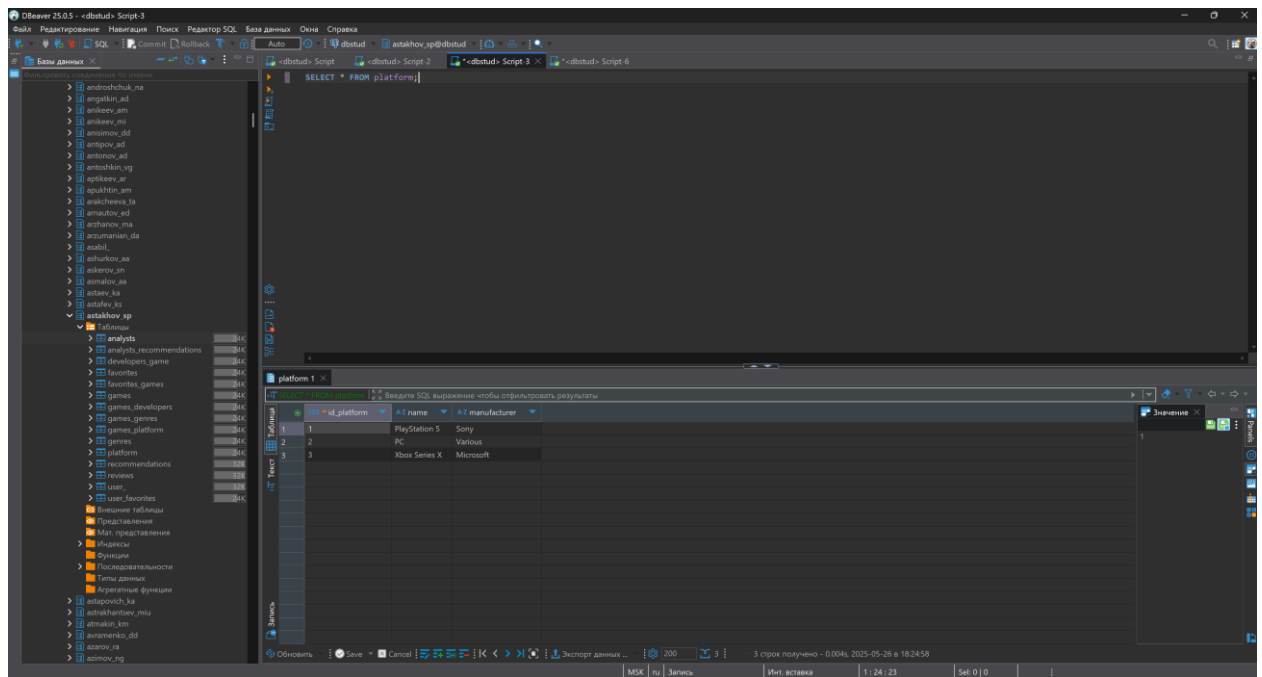


Рисунок 11 - Вывод данных в таблице

3. Вывод

В ходе практической работы, мы сумели скачать программное обеспечение для работы с PostgreSQL, создали базу данных, таблицы в ней, а также поработали с вводом и выводом значений этих таблиц.